

Entrega UT5

Actividad 1: Diseño de una Clase en POO Descripción: Diseñar una clase en Java, incluyendo atributos, métodos, y aplicar herencia y polimorfismo. Deben entregar el código y una breve explicación del diseño.

Codigo:

public abstract class Vehiculo

```
{
    private int matricula;
    private String modelo;
    private String color;
    private int ruedas;

    public Vehiculo(int matricula, String modelo, String color, int ruedas)
    {
        this.matricula = matricula;
        this.modelo = modelo;
        this.color = color;
        this.ruedas = ruedas;
    }

    public int getMatricula()
    {
        return matricula;
    }

    public void setMatricula(int matricula)
    {
        this.matricula = matricula;
    }

    public String getModelo()
    {
        return modelo;
    }

    public void setModelo(String modelo)
    {
        this.modelo = modelo;
    }

    public String getColor()
    {
        return color;
    }
}
```

```

    public void setColor(String color)
    {
        this.color = color;
    }

    public int getRuedas()
    {
        return ruedas;
    }

    public void setRuedas(int ruedas)
    {
        this.ruedas = ruedas;
    }

    public void acelerar()
    {
        System.out.println("El vehiculo está acelerando.");
    }

    public abstract void descripcion();
}

public class Coche extends Vehiculo
{
    public Coche(int matricula, String modelo, String color, int ruedas)
    {
        super(matricula, modelo, color, ruedas);
    }

    @Override
    public void descripcion()
    {
        System.out.println("Es un Coche.");
    }
}

```

Explicación->

He creado una clase abstracta en Java llamada Vehiculo, de la cual derivan otras clases.

Esta clase contiene los atributos matrícula, modelo, color y ruedas, definidos como private para aplicar encapsulación, evitando el acceso directo desde otras clases. Para acceder a estos atributos se utilizan getters y setters.

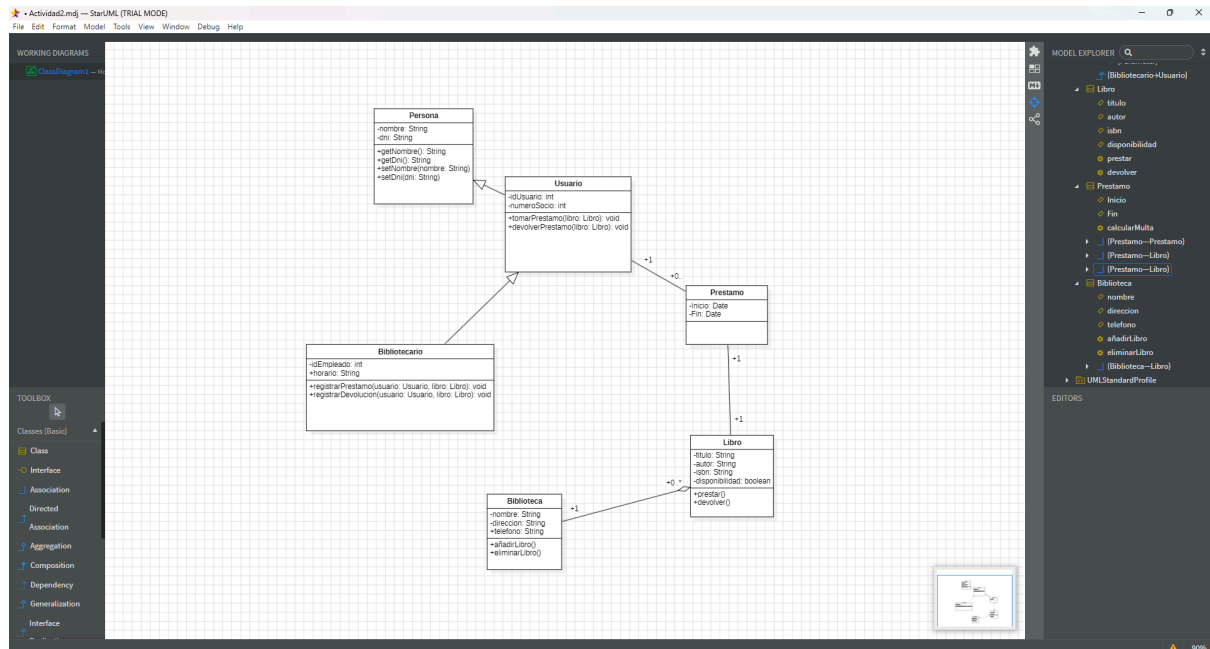
Además, la clase dispone de un constructor para inicializar los atributos.

He definido un método público acelerar() común a todos los vehículos, y un método abstracto descripción(), que debe ser implementado por las clases hijas. Esto permite aplicar polimorfismo, ya que cada clase derivada puede proporcionar su propia implementación de este método.

Por otro lado, se ha creado la clase Coche, que hereda de Vehiculo mediante extends. Esta clase reutiliza el comportamiento común definido en la clase base y sobrescribe el método descripcion().

De este modo consigo que usando una referencia del tipo Vehiculo, se ejecute el comportamiento específico de cada subclase en tiempo de ejecución.

Actividad 2: Creación de un Diagrama de Clases UML Descripción: Crear un diagrama de clases UML que represente un sistema sencillo (por ejemplo, gestión de biblioteca), incluyendo clases, atributos, métodos y relaciones (asociación, agregación, composición, herencia).



Actividad 3: Ingeniería Directa con Enterprise Architect Descripción: Utiliza Enterprise Architect (u otra herramienta similar) para generar código automáticamente a partir de un diagrama UML previamente creado.

Archivos compartidos en la carpeta

Actividad 4: Ingeniería Inversa desde Código Descripción: Mediante un fragmento de código existente (puedes utilizar el del ej1) genera el diagrama UML correspondiente, identificando clases, relaciones y estructuras

